

Bài 3: LỚP VÀ CÁC THÀNH PHẦN CỦA LỚP ĐỐI TƯỢNG (Số tiết: 5 tiết)

Mục tiêu

- Hiểu được lớp đối tượng và các thành phần của lớp
- Sử dụng được Java để định nghĩa lớp và các thành phần của lớp
- Sử dụng được ngôn ngữ java để làm việc với các đối tượng của lớp
- Hiểu được nguyên lý bao bọc và che dấu thông tin áp dụng cho lớp

Nội dung chính

- Cấu trúc lớp và khai báo/Định nghĩa lớp đối tượng
- Định nghĩa các thành phần của lớp
- Phạm vi truy cập
- Toán tử tạo lập

3.1. Cấu trúc lớp và khai báo các thành phần của lớp

Lớp là mô hình khái quát đặc tính của một họ thực thể vật lý hoặc trừu tượng trong chương trình.

Định nghĩa lớp là đặc tả một kiểu dữ liệu mới và mô tả cách cài đặt kiểu dữ liệu đó.

Mỗi đối tượng là một thể hiện của lớp.

Cú pháp định nghĩa lớp

```
[<Phạm vi hoặc thuộc tính kiểm soát truy nhập>] class <Tên lớp> [extends <Tên lớp cha>] [implements <Tên giao diện>]{  
    <Các thành phần của lớp>  
}
```

Giải thích:

- *Phạm vi*: Xác định phạm vi hoặc kiểm soát truy nhập:
- *abstract*: Không thể tạo ra một đối tượng cụ thể của lớp, lớp loại này là khung cơ sở nhất để dẫn xuất ra các lớp con cháu.
- *public*: (công khai) Cho phép sử dụng mọi nơi trong hệ thống.
- *protected*: cho phép truy cập trong gói và trong các lớp thuộc gói khác nhưng được kế thừa.
- *private*: chỉ được sử dụng bên trong lớp, không cho phép truy cập và kế thừa đối với các lớp khác ở trong cùng gói và khác gói.
- *final*: (hằng) nếu nó đứng trước khai báo lớp thì nó không được sử dụng để kế thừa cho các lớp con, nếu nó đứng trước khai báo biến thì nó là “hằng” (chỉ được gán giá

trị một lần duy nhất), nếu nó đứng trước các phương thức thì nó sẽ không được phép viết đè ở các lớp con.

- Nếu không khai báo (public, protected, private) thì gọi là các thành phần “mặc định”, khi đó phạm vi là truy cập và kế thừa là trong gói chứa lớp đó.
- *tên_lớp_cha*: Tên lớp cha được kế thừa
- *DS_tên_các_giao_diện*: Tên các giao diện (được viết phân cách nhau bởi dấu “,”) được phép sử dụng các phương thức và các hằng.
- *class*, *extends*, *implements* là các từ khoá.

Những phần trong cặp [và] là tùy chọn(có thể có hoặc có thể không).

<Các thành phần của lớp> bao gồm:

- Các biến dữ liệu thành phần(attributes): Xác định các thuộc tính (dữ liệu) của lớp, thông thường dùng để mô tả các đặc tính của đối tượng thuộc lớp đó
- Các toán tử tạo lập(constructor): Xác định cách khởi tạo một đối tượng thuộc lớp
- Các phương thức(method): Xác định hành vi của các đối tượng thuộc lớp

3.2 Định nghĩa các thành phần của lớp

3.2.1 Các thuộc tính dữ liệu của lớp

Thuộc tính dữ liệu của lớp là các biến thành phần có thể thuộc kiểu nguyên thủy, kiểu tham chiếu.

Khai báo:

<Phạm_vi> <Kiểu> <tên_biến>;

<Phạm_vi> <Kiểu> <tên_biến> = <Giá_trị_khởi_tạo>;

- *Phạm vi*: xác định phạm vi truy cập của các đối tượng đến thuộc tính. có thể chọn một trong các phạm vi:
- *static*: Khi một thuộc tính được khai báo static thì khi đó thuộc tính đó là “sở hữu” của tất cả các đối tượng của lớp đó chứ không phải nó là của riêng của từng đối tượng thuộc lớp đó.
- *public*: Có thể truy cập mọi nơi trong hệ thống (các đối tượng khác lớp có thể truy cập được)
- *protected*: cho phép truy cập trong gói và trong các lớp thuộc gói khác nhưng được kế thừa.
- *private*: của riêng đối tượng chỉ bên trong đối tượng.
- *final*: Hằng (chỉ được khởi tạo giá trị một lần duy nhất)
- *volatile*: là biến được lưu trữ trên bộ nhớ chính, được dùng với luồng kết hợp với từ khóa synchronized.

3.2.2 Định nghĩa hàm thành phần

Hành vi của các đối tượng thuộc một lớp được xác định bởi các hàm thành phần của lớp đó, hành vi thể hiện “vai trò” của đối tượng đó trong hệ thống.

Là các hàm, thủ tục thực hiện các phép xử lý trên các biến thành phần.

Khai báo

<Phạm_vi> <Kiểu_giá_trị> <tên_hàm>(<danhsách các tham số hình thức>)
<các_mệnh_đề_throws> {//Thân hàm}

Phạm_vi:

- *public*: Cho phép truy cập mọi nơi trong hệ thống
- *private*: Chỉ được truy xuất (gọi) bên trong lớp chứa nó
- *protected*: lớp đó và lớp con của nó được truy xuất
- *final*: Không cho phép nạp chồng và ghi đè

Các tính chất:

- *static*: Nó có ý nghĩa tương tự như với thuộc tính *static*, thông thường chỉ những phương thức chung và không dùng cho một đối tượng cụ thể thì khi đó phương thức đó nó mới là phương thức tĩnh (*static*).
- *abstract*: Phương thức trừu tượng(là phương thức “mẫu”, nó chỉ nói lên là nó làm được gì mà không mô tả nó làm như thế nào), do đó nó không cài đặt nội dung ở lớp khai báo nó
- *native*: dùng cho phương thức khi cài đặt phụ thuộc môi trường trong một ngôn ngữ khác, như C hay hợp ngữ.
- *synchronized*: dùng trong lập trình đa luồng, nhằm ngăn các tác động của các phương thức khác của các luồng khác khi phương thức đang được thực hiện.
- Giá trị mặc định là *public*.
<Kiểu trả lại> có thể là kiểu nguyên thủy, kiểu lớp hoặc không có giá trị trả lại (kiểu *void*).

<Danhsách tham biến hình thức> bao gồm dãy các tham biến (kiểu và tên) phân cách với nhau bởi dấu phẩy.

Sử dụng phương thức:

Nếu phương thức không phải là static thì cần phải tạo ra một đối tượng cụ thể và gọi phương thức thông qua tên đối tượng như cấu trúc sau:

<tên đối tượng>.<tên_hàm>(< danhsách các tham số hiện thời>)

Khai báo đối tượng: <tên lớp> <tên đối tượng> ;

Khởi tạo giá trị thuộc tính cho đối tượng khi sử dụng Toán tử tạo lập lập(constructor) tự định nghĩa :

<tên đối tượng> = new <tên lớp> (<danhsách tham số hiện thời >) ;

Khai báo và khởi tạo đối tượng giá trị:

<tên lớp> <tên đối tượng> = new <tên lớp> (<danhsách tham số hiện thời >);

- Nếu phương thức là static thì gọi thông qua tên lớp hoặc thông qua tên đối tượng (nên gọi các phương thức tĩnh thông qua tên lớp)

<tên_lớp>.<tên_hàm_tĩnh> (<danh sách tham số hiện thời >)

- Các tham số trong các lời gọi hàm cung cấp cách thức trao đổi thông tin giữa đối tượng gửi và đối tượng nhận thông điệp.
 - Số các tham biến của danh sách hiện thời phải bằng số các tham biến của danh sách tham biến hình thức.
 - Kiểu của các tham biến hiện thời phải tương thích với kiểu của tham biến hình thức tương ứng.

Ví dụ 1: Tập Demo.java

```
public class Demo {
    public static void main(String args[]) {
        int i = 5;
        if (i == 0) {
            return;
        }
        output(checkValue(args.length));
    }
    static void output(int value) {
        System.out.println(value);
    }
    static int checkValue(int i) {
        if (i > 3) {
            return 1;
        } else {
            return 2;
        }
    }
}
```

Ví dụ 2: Tập Demo2.java

```
public class Demo2 {
    public static void main(String args[]) {
```

```

    int i = 5;

    Demo2 d = new Demo2();

    if (i == 0) {

        return;

    }

    output(d.checkValue(args.length));

}

static void output(int value) {

    System.out.println(value);

}

public int checkValue(int i) {

    if (i > 3) {

        return 1;

    } else {

        return 2;

    }

}

}

```

***Phương thức và thuộc tính static:** Phương thức và thuộc tính độc lập với đối tượng, có thể sử dụng mà không cần thông qua đối tượng, Phương thức tĩnh: không sử dụng được thuộc tính và phương thức “thông thường” (non-static) mà nó chỉ được gọi đến các phương thức và thuộc tính tĩnh khác

3.2.3 Truyền tham số và lời gọi hàm

Truyền các giá trị kiểu nguyên thủy

Trong Java, khi gọi một phương thức và truyền các giá trị nguyên thủy cho phương thức, được gọi là truyền tham trị. Việc thay đổi giá trị chỉ có hiệu lực trong phương thức được gọi, không có hiệu lực bên ngoài phương thức.

Các tham số có kiểu dữ liệu nguyên thủy là byte, short, int, long, float, double, boolean, char.

Ví dụ 3: *Truyền các giá trị nguyên thủy*

```

class KháchHang1 { // Lớp khách hàng

    public static void main(String[] arg) {

```

```

        HangSX banh = new HangSX(); // Tạo ra một đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không
        đổi

        System.out.println("Tien ban duoc : " + tien);
    }
}
// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, double gia) {
        gia = gia / 2;
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng
        tới giaBan, số tiền bị thay đổi theo
    }
}

```

Truyền các giá trị tham chiếu đối tượng.

Trong Java, khi gọi một phương thức và truyền một tham chiếu cho phương thức, được gọi là truyền tham chiếu. Việc thay đổi giá trị của biến tham chiếu bên trong phương thức làm thay đổi giá trị của nó.

Trong Java, tất các phương thức có tham số là biến có kiểu là các lớp (class) đều là kiểu tham chiếu

//KhachHang2.java

```

class KhachHang2 { // Lớp khách hàng
    public static void main(String[] arg) {
        Banh banhMoi = new Banh(); // Tạo ra một đối tượng (1)
        System.out.println("Nhoi thit vao banh truoac khi nuong:" +
        banhMoi.thit);
        nuong(banhMoi); // (2)
        System.out.println("Thit cua banh sau khi nuong:" + banhMoi.thit);
    }
    public static void nuong(Banh banhNuong) { // (3)

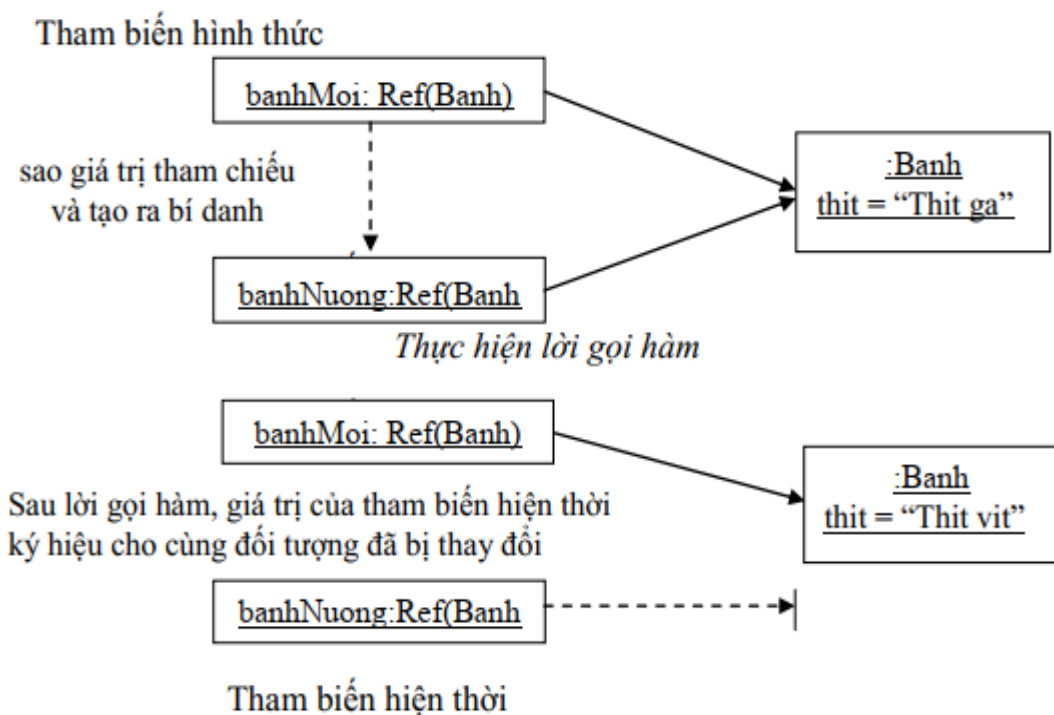
```

```

        banhNuong.thit = "Thịt vịt
                "; // đổi nhân thành thịt vịt
        banhNuong = null; // (4)
    }
}

class Banh { // Lớp Banh (5)
    String thit = "Thịt gà
                "; // Quy định của hãng làm nhân bánh bằng thịt gà
}

```



Hình 3.1 Truyền các giá trị tham chiếu đối tượng

Truyền tham chiếu: đối với đối tượng thì nội dung của tham chiếu (LValue) được copy lên stack.

Truyền đối số trong dòng lệnh

```

class Pass {
    public static void main(String parameters[]) {
        System.out.println("This is what the main method
received");
        System.out.println(parameters[0]);
        System.out.println(parameters[1]);
        System.out.println(parameters[2]);
    }
}

```

```
}  
  
}
```

Biên dịch chương trình: **javac PassArgument.java**

Thực thi chương trình với dòng lệnh: **java PassArgument A 123 B1**

Sẽ thu được trên màn hình kết quả:

```
This is what the main method received  
  
A  
  
123  
  
B1
```

Các tham biến final

Tham biến loại này được gọi là biến cuối “trắng”, nghĩa là nó không được khởi tạo giá trị (là trắng) cho đến khi nó được gán một trị nào đó và khi đã được gán trị thì giá trị đó là cuối cùng, không thay đổi được.

Ví dụ 4: Sử dụng tham biến final

```
//KhachHang3.java
```

```
class KhachHang3 { // Lớp khách hàng
```

```
    public static void main(String[] arg) {
```

```
        HangSX banh = new HangSX(); // Tạo ra 1 đối tượng
```

```
        int giaBan = 20;
```

```
        double tien = banh.tinh(10, giaBan);
```

```
        System.out.println("Gia ban: " + giaBan); // giaBan không  
        đổi
```

```
        System.out.println("Tien ban duoc : " + tien);
```

```
    }
```

```
}
```

```
// Lớp Hãng sản xuất
```

```
class HangSX {
```

```
    double tinh(int num, final double gia) { // (1)
```

```
        gia = gia / 2.0; // (2)
```



```

        return num * gia;// Thay đổi gia nhưng không ảnh hưởng
        tới giaBan, số tiền vẫn bị thay đổi theo
    }
}

```

3.3 Phạm vi truy cập

Khi xây dựng một lớp ta có thể hạn chế sự truy cập đến các thành viên của lớp, từ một đối tượng khác. Ta tóm tắt qua bảng sau:

Bảng Error! No text of specified style in document..1 Thuộc tính kiểm soát truy nhập các thành phần của lớp

Từ khoá	Truy cập trong chính lớp đó	Truy cập trong lớp con cùng gói	Truy cập trong lớp con khác gói	Truy cập trong lớp khác cùng gói	Truy cập trong lớp khác khác gói
private	X	-	-	-	-
protected	X	X	X	X	-
public	X	X	X	X	X
default	X	X	-	X	-

Trong bảng trên thì X thể hiện cho sự truy cập hợp lệ còn – thể hiện không thể truy cập vào thành phần này.

Ví dụ 1: Các thành phần private

Các thành viên private chỉ có thể sử dụng bên trong lớp, ta không thể truy cập các thành viên private từ bên ngoài lớp này.

```

class Alpha {
    private int iamprivate;
    private void privateMethod() {
        System.out.println("privateMethod");
    }
}

class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
    }
}

```

```

        a.iamprivate = 10;// không hợp lệ
        a.privateMethod();// không hợp lệ }
    }
}

```

Ví dụ 2: Các thành phần public

Các thành viên public có thể truy cập từ bất cứ đâu, ta sẽ xem ví dụ sau:

```

public class Alpha {
    public int iampublic;
    public void publicMethod() {
        System.out.println("publicMethod");
    }
}

package Roman;
import Greek.*;
class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iampublic = 10;// hợp lệ
        a.publicMethod();// hợp lệ
    }
}

```

Ví dụ 3: các thành phần có mức truy xuất gói:

Khi ta khai báo các thành viên mà không sử dụng một trong các từ public, private, protected thì java mặc định thành viên đó có mức truy cập gói.

```

class Alpha {
    int iampackage;
    void packageMethod() {
        System.out.println("packageMethod");
    }
}

```

```

    }

    package Greek;

    class Beta {

        void accessMethod() {

            Alpha a = new Alpha();

            a.iampackage = 10; // hợp lệ

            a.packageMethod(); // hợp lệ

        }

    }

```

3.4 Toán tử tạo lập

Mục đích chính của toán tử tạo lập (constructor): Đặt các giá trị khởi tạo cho các đối tượng khi một đối tượng được tạo ra bằng toán tử new.

Khai báo: [<phạm vi>] <Tên lớp> ([<danh sách tham biến>]) {
 // Nội dung cần tạo lập
 }

Trong đó:

- Phạm_vi có thể là: public, protected, private
- Phần thân là các lệnh khởi tạo giá trị các thuộc tính của đối tượng, là thành phần tùy chọn.
- Danh sách các tham số : tùy chọn

Phân loại:

- Toán tử tạo lập mặc định không tường minh: **Tên_Lớp()** {}. Khi định nghĩa một lớp mà không xây dựng toán tử tạo lập nào thì hệ thống sẽ cung cấp toán tử tạo lập mặc định không tường minh.
- Toán tử tạo lập mặc định tường minh:
 TênLớp(<danh sách tham biến hình thức>) {
 tên thuộc tính= giá trị;
 }

Giá trị của các biến thành phần được khởi tạo không mặc định bằng các phép gán trong thân của toán tử bằng các giá trị được truyền vào.

Ví dụ 5: Minh họa một phương thức khởi tạo của lớp Person bằng cách gán giá trị cho các thuộc tính tên và tuổi.

```

class Person {
    public String name;
    public int age;
    // Phương thức tạo lập mặc định không tường minh
    // public Person(){}
    // Phương thức tạo lập mặc định tường minh
    public Person(){
        name = "";
        age = 16;
    }
    // Phương thức tạo lập không mặc định
    public Person(String name1, int age1){
        name = name1;
        age = age1;
    }
    // Phương thức tạo lập không mặc định
    public Person(String name1, int age1){
        name = name1;
        age = age1;
    }
    /*public Person(String name, int age){
        this.name = name;
        this.age = age;
    }*/
    public void show(){
        System.out.println( name + " is " + age + " years old!");
    }
}

```

Nạp chồng các toán tử tạo lập

Giống như các hàm thành phần, các toán tử tạo lập có thể nạp chồng với nhiều nội dung thực hiện khác nhau.

Khi tạo lập đối tượng sử dụng toán tử tạo lập nào thì toán tử đó được gọi.

Ví dụ 6:

```
class BongDen{
    // Biến thành phần (1)
    private int soWatts;
    private boolean batTat;
    private String viTri;
    // Định nghĩa toán tử tạo lập số 1 (2)
    BongDen(){
        this(40, true);
        System.out.println("Toán tử số 1"); }
    // Định nghĩa toán tử tạo lập số 2 (3)
    BongDen(int w, boolean s){
        this(w, s, "XX");
        System.out.println("Toán tử số 2"); }
    //Định nghĩa toán tử tạo lập số 3 (4)
    BongDen(int soWatts, boolean batTat, String viTri){
        this.soWatts = soWatts;
        this.batTat = batTat;
        this.viTri = new String(viTri);
        System.out.println("Toán tử số 3"); }
}

class DenTuyp extends BongDen {
    private int doDai;
    private int mau;
    DenTuyp(int leng, int colo){
        this(leng,colo,100,true,"Chua biet");
    }
}
```

```

        DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){
            super(soWatt, bt, noi);
            this.doDai = leng;
            this.mau = colo;
        }
    }

    public class NhaKho{
        public static void main(String args[]){
            System.out.println("Tao ra bong đèn tuyp");
            DenTuyp d = new DenTuyp(20, 5);
        }
    }

```

3.5 Tạo và sử dụng đối tượng

Trong Java, đối tượng là một thực thể của một lớp (class). Một lớp định nghĩa các thuộc tính (attributes) và phương thức (methods), và đối tượng là phiên bản cụ thể của lớp đó, chứa dữ liệu cụ thể và có thể thực hiện các hành động do lớp định nghĩa. Để tạo và sử dụng đối tượng, đầu tiên cần định nghĩa một lớp. Một lớp là bản thiết kế mô tả các thuộc tính và hành vi của các đối tượng.

Ví dụ 1: Khai báo lớp *Car* với các thuộc tính *make* và *model*, và một phương thức *displayInfo* để in thông tin của xe.

```

public class Car {
    // Thuộc tính
    private String make;
    private String model;
    private int year;

    // Constructor (Phương thức khởi tạo)
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
}

```

```

    }

    // Phương thức hiển thị thông tin xe

    public void displayInfo() {
        System.out.println("Make: " + make);
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
}

```

Để tạo một đối tượng từ lớp, ta sử dụng từ khóa *new* và gọi phương thức khởi tạo (constructor).

```
Car myCar = new Car("Toyota", "Corolla", 2020);
```

Khi đó:

- *myCar* là đối tượng của lớp *Car*.
- *new Car("Toyota", "Corolla", 2020)* tạo một đối tượng mới và khởi tạo các thuộc tính của nó với các giá trị *make* = "Toyota", *model* = "Corolla", và *year* = 2020.

Sử dụng đối tượng: Sau khi tạo đối tượng, bạn có thể gọi các phương thức và truy cập các thuộc tính của đối tượng đó.

```
myCar.displayInfo();
```

Kết quả:

Make: Toyota

Model: Corolla

Year: 2020

Ta cũng có thể truy cập và cập nhật các thuộc tính của đối tượng (nếu chúng không có từ khóa *private*):

```
myCar.year = 2021; // Thay đổi năm sản xuất
myCar.displayInfo();
```

Ví dụ 2:

```

public class Car {
    String make;
    String model;
    int year;
}

```

```

public Car(String make, String model, int year) {
    this.make = make;
    this.model = model;
    this.year = year;
}

public void displayInfo() {
    System.out.println("Make: " + make);
    System.out.println("Model: " + model);
    System.out.println("Year: " + year);
}

public static void main(String[] args) {
    Car myCar = new Car("Toyota", "Corolla", 2020);
    myCar.displayInfo();
    // Thay đổi thuộc tính
    myCar.year = 2021;
    System.out.println("Updated Year: " + myCar.year);
}
}

```

Kết quả:

Make: Toyota

Model: Corolla

Year: 2020

Updated Year: 2021

Tổng kết bài học

Bài học này cung cấp kiến thức cơ bản về lớp đối tượng trong lập trình hướng đối tượng, cùng với các thành phần cấu thành và cách sử dụng lớp. Nội dung bao gồm:

1. Lớp đối tượng: Giới thiệu về lớp (class), là khuôn mẫu để tạo ra các đối tượng, chứa các thuộc tính và phương thức của đối tượng.
 - Định nghĩa lớp đối tượng và các thành phần của lớp: Định nghĩa lớp và các thành phần cấu thành như thuộc tính dữ liệu (data attributes) và các phương thức (methods).

- Các thuộc tính dữ liệu của lớp: Giải thích các thuộc tính lưu trữ trạng thái của đối tượng, các biến thành phần đại diện cho dữ liệu của lớp.
 - Định nghĩa hàm thành phần: Giới thiệu cách viết các phương thức, là các hàm xác định hành vi của đối tượng.
 - Truyền tham số và lời gọi hàm: Giải thích cách truyền tham số vào phương thức và cách gọi phương thức của đối tượng.
 - Toán tử tạo lập đối tượng: Cách tạo đối tượng mới từ lớp thông qua toán tử đặc biệt.
2. Các thuộc tính kiểm soát truy nhập các thành phần của lớp: Đề cập đến các mức độ truy cập khác nhau của các thành phần trong lớp (như private, protected, public), giúp kiểm soát khả năng truy cập và bảo vệ dữ liệu.
 3. Tạo và sử dụng đối tượng: Hướng dẫn cách khởi tạo đối tượng từ lớp, cách sử dụng các phương thức và truy cập các thuộc tính của đối tượng để thực hiện các thao tác cụ thể.

Chương này cung cấp nền tảng cần thiết để hiểu cách làm việc với các lớp và đối tượng, cũng như quản lý các thành phần của chúng, là những kiến thức cốt lõi của lập trình hướng đối tượng.

Bài tập thực hành

Bài tập 1: Quản lý nhân sự

Mô tả: Một công ty cần hệ thống quản lý nhân sự để theo dõi thông tin của các nhân viên, bao gồm tên, mã nhân viên, chức vụ, và lương. Hệ thống cũng cần có khả năng tính toán lương tháng dựa trên các yếu tố như ngày làm việc, số giờ làm thêm và các khoản thưởng phạt.

Gợi ý giải quyết:

- **Lớp:** Employee (Nhân viên)
- **Thuộc tính:** employeeID, name, position, salary, workDays, overtimeHours, bonus, penalty.
- **Phương thức:** calculateMonthlySalary(), addWorkDays(), addOvertimeHours().

Bài tập 2: Quản lý tài khoản ngân hàng

Mô tả: Ngân hàng cần một hệ thống để quản lý tài khoản của khách hàng, bao gồm việc mở tài khoản, nạp tiền, rút tiền, và kiểm tra số dư. Mỗi khách hàng có thể có nhiều tài khoản với số dư khác nhau.

Gợi ý giải quyết:

- **Lớp:** BankAccount (Tài khoản ngân hàng), Customer (Khách hàng).
- **Thuộc tính:** accountNumber, balance, accountType cho BankAccount; customerID, name, accounts cho Customer.
- **Phương thức:** deposit(), withdraw(), checkBalance() cho BankAccount; addAccount() cho Customer.

Bài tập 3: Quản lý cửa hàng sách

Mô tả: Một cửa hàng sách cần hệ thống để quản lý sách, tác giả, và đơn hàng. Hệ thống cần lưu trữ thông tin sách, theo dõi số lượng sách bán ra, và quản lý đơn hàng của khách hàng.

Gợi ý giải quyết:

- **Lớp:** Book (Sách), Author (Tác giả), Order (Đơn hàng).
- **Thuộc tính:** bookID, title, author, price, stock cho Book; authorID, name, booksWritten cho Author; orderID, customer, booksOrdered cho Order.
- **Phương thức:** sellBook(), updateStock() cho Book; addBook() cho Author; calculateTotalPrice() cho Order.

Bài tập 4: Quản lý hệ thống đặt phòng khách sạn

Mô tả: Một khách sạn cần một hệ thống để quản lý đặt phòng, khách hàng và thanh toán. Hệ thống phải có khả năng quản lý các phòng trống, kiểm tra tình trạng phòng, và xử lý đặt phòng cũng như thanh toán.

Gợi ý giải quyết:

- **Lớp:** Room (Phòng), Customer (Khách hàng), Booking (Đặt phòng).
- **Thuộc tính:** roomID, roomType, price, availability cho Room; customerID, name, phone cho Customer; bookingID, room, customer, checkInDate, checkOutDate cho Booking.
- **Phương thức:** checkAvailability(), bookRoom() cho Room; makePayment() cho Booking.

Bài tập 5: Quản lý hệ thống học trực tuyến

Mô tả: Một tổ chức giáo dục cần một hệ thống để quản lý các khóa học trực tuyến, học viên và giảng viên. Hệ thống cần lưu trữ thông tin về khóa học, danh sách học viên đăng ký và điểm số của họ.

Gợi ý giải quyết:

- **Lớp:** Course (Khóa học), Student (Học viên), Instructor (Giảng viên).
- **Thuộc tính:** courseID, courseName, instructor, students cho Course; studentID, name, coursesEnrolled cho Student; instructorID, name, coursesTaught cho Instructor.
- **Phương thức:** enrollStudent(), assignInstructor(), recordGrade() cho Course; registerCourse() cho Student.